Parallel.Project.ProjectInitializations.Backend Initialization Guide: NetBeans Java Projects

Project: Online Marketplace System (CSE 352)
Last Updated: April 30, 2026
Target OS: Windows
NetBeans Version: 24+
Java Version: JDK 21 or 23

Table of Contents

1. Prerequisites
2. Project Overview
3. Shared Models Project
4. Socket Server Project
5. Spring Boot Web Services Project
6. Building All Projects
7. Running the Servers
8. Ports Reference
9. Configuration for CI/CD
10. Troubleshooting

---

1. Prerequisites

1.1 Install JDK 21 or 23

1. Download from Adoptium or Oracle
2. Install to default location: `C:\Program Files\Java\jdk-21`
3. Set `JAVA_HOME` environment variable:
   - Open System Properties → Environment Variables
   - Add `JAVA_HOME = C:\Program Files\Java\jdk-21`
   - Add `%JAVA_HOME%\bin` to `Path`

Verify:

```
java -version
javac -version
```

1.2 Install NetBeans 24+

1. Download from Apache NetBeans
2. Run installer (select **Java SE** bundle)
3. Launch NetBeans

1.3 Configure NetBeans to use JDK 21

1. `Tools` → `Java Platforms` → `Add Platform`
2. Select `C:\Program Files\Java\jdk-21`
3. Set as default

1.4 Maven (bundled with NetBeans)

NetBeans includes Maven. No separate installation needed.

---

2. Project Overview

You will create **three separate Maven projects** in NetBeans:

#	Project Name	Type	Purpose
1	`MarketplaceSharedModels`	Java Library (JAR)	Shared entities, DTOs, exceptions
2	`MarketplaceSocketServer`	Java Application	Core TCP socket server (port 9090)
3	`MarketplaceWebServices`	Spring Boot Web	REST API (port 8080)

Dependency Flow:

```
MarketplaceSharedModels (JAR)
          ▲
          │ (dependency)
    ┌─────┴─────┐
Socket Server  Web Services
```

Directory Structure on Disk:

```
C:\Users\YourName\Documents\OnlineMarketplace\
│
├── MarketplaceSharedModels\
```

```
├── MarketplaceSocketServer\
└── MarketplaceWebServices\
```

---

3. Shared Models Project

3.1 Create Project

1. `File` → `New Project`
2. Category: **Maven** → Project: **Java Library** → Next

Field	Value
Project Name	`MarketplaceSharedModels`
Project Location	`C:\Users\YourName\Documents\OnlineMarketplace`
Group ID	`com.marketplace`
Version	`1.0-SNAPSHOT`

Click **Finish**

3.2 Create Package Structure

Right-click `Source Packages` → `New` → `Java Package`:

Package	Purpose
`com.marketplace.models`	Entity classes (User, Item, Transaction, Inventory)
`com.marketplace.dto`	DTOs (UserResponseDto, ItemDto, TransactionDto)
`com.marketplace.exceptions`	Custom exceptions

3.3 Create Entity Classes

`com.marketplace.models.User.java`

```java
package com.marketplace.models;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class User {
    private String userId;
    private String username;
    private String email;
    private String passwordHash;
    private BigDecimal balance;
```

```
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    // Constructors
    public User() {}

    public User(String userId, String username, String email, String
passwordHash) {
        this.userId = userId;
        this.username = username;
        this.email = email;
        this.passwordHash = passwordHash;
        this.balance = BigDecimal.ZERO;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    // Getters and Setters (generate in NetBeans: Right-click → Insert Code
→ Getter/Setter)
    // ...
}
```

`com.marketplace.models.Item.java`

```
package com.marketplace.models;

import java.math.BigDecimal;

public class Item {
    private String itemId;
    private String sellerId;
    private String name;
    private String brand;
    private BigDecimal price;
    private int quantity;
    private boolean isAvailable;

    // Constructors
    public Item() {}

    public Item(String itemId, String sellerId, String name, String brand,
BigDecimal price, int quantity) {
        this.itemId = itemId;
        this.sellerId = sellerId;
        this.name = name;
        this.brand = brand;
        this.price = price;
        this.quantity = quantity;
        this.isAvailable = true;
    }

    // Getters and Setters
    // ...
}
```

`com.marketplace.models.Transaction.java`

```java
package com.marketplace.models;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class Transaction {
    private String transactionId;
    private String buyerId;
    private String sellerId;
    private String itemId;
    private BigDecimal amount;
    private LocalDateTime timestamp;
    private String status;  // COMPLETED, FAILED, PENDING

    // Constructors, Getters, Setters
}
```

`com.marketplace.models.Inventory.java`

```java
package com.marketplace.models;

public class Inventory {
    private String sellerId;
    private String itemId;
    private int stockQuantity;

    // Constructors, Getters, Setters
}
```

3.4 Create DTOs

`com.marketplace.dto.UserResponseDto.java`

```java
package com.marketplace.dto;

import java.math.BigDecimal;

public class UserResponseDto {
    private String userId;
    private String username;
    private String email;
    private BigDecimal balance;

    // Constructors, Getters, Setters
}
```

`com.marketplace.dto.ItemDto.java`

```java
package com.marketplace.dto;

import java.math.BigDecimal;
```

```java
public class ItemDto {
    private String itemId;
    private String sellerId;
    private String name;
    private String brand;
    private BigDecimal price;
    private int quantityAvailable;

    // Constructors, Getters, Setters
}
```

`com.marketplace.dto.TransactionDto.java`

```java
package com.marketplace.dto;

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class TransactionDto {
    private String transactionId;
    private String buyerId;
    private String sellerId;
    private String itemId;
    private BigDecimal amount;
    private LocalDateTime timestamp;
    private String status;

    // Constructors, Getters, Setters
}
```

3.5 Create Custom Exceptions

`com.marketplace.exceptions.InsufficientBalanceException.java`

```java
package com.marketplace.exceptions;

public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

`com.marketplace.exceptions.ItemNotFoundException.java`

```java
package com.marketplace.exceptions;

public class ItemNotFoundException extends Exception {
    public ItemNotFoundException(String message) {
        super(message);
    }
}
```

`com.marketplace.exceptions.UserNotFoundException.java`

```java
package com.marketplace.exceptions;

public class UserNotFoundException extends Exception {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

`com.marketplace.exceptions.UnauthorizedActionException.java`

```java
package com.marketplace.exceptions;

public class UnauthorizedActionException extends Exception {
    public UnauthorizedActionException(String message) {
        super(message);
    }
}
```

3.6 Build and Install to Local Maven Repository

1. Right-click `MarketplaceSharedModels` project
2. Select **Clean and Build**
3. Right-click again → **Run Maven** → **install**

Success message:

```
BUILD SUCCESS
[INFO] Installing ... to
C:\Users\YourName\.m2\repository\com\marketplace\...
```

---

4. Socket Server Project

4.1 Create Project

1. `File` → `New Project`
2. Category: **Maven** → Project: **Java Application** → Next

Field	Value
Project Name	`MarketplaceSocketServer`
Project Location	`C:\Users\YourName\Documents\OnlineMarketplace`
Group ID	`com.marketplace`

Field	Value
Version	`1.0-SNAPSHOT`
Package	`com.marketplace.socket`

Uncheck "Create Main Class" (we will create manually)

Click **Finish**

4.2 Configure `pom.xml`

Replace the generated `pom.xml` with:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.marketplace</groupId>
    <artifactId>MarketplaceSocketServer</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <!-- Shared Models (local JAR) -->
        <dependency>
            <groupId>com.marketplace</groupId>
            <artifactId>MarketplaceSharedModels</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>

        <!-- PostgreSQL JDBC Driver (for CockroachDB) -->
        <dependency>
            <groupId>org.postgresql</groupId>
            <artifactId>postgresql</artifactId>
            <version>42.7.3</version>
        </dependency>

        <!-- JWT for authentication -->
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.12.5</version>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
```

```
            <artifactId>jjwt-impl</artifactId>
            <version>0.12.5</version>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-jackson</artifactId>
            <version>0.12.5</version>
            <scope>runtime</scope>
        </dependency>

        <!-- JSON parsing -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.10.1</version>
        </dependency>

        <!-- Dotenv for environment variables (CI/CD) -->
        <dependency>
            <groupId>io.github.cdimascio</groupId>
            <artifactId>dotenv-java</artifactId>
            <version>3.0.0</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.12.1</version>
            </plugin>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.1</version>
                <configuration>
                    <archive>
                        <manifest>

<mainClass>com.marketplace.socket.Main</mainClass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

4.3 Create Package Structure

Right-click `Source Packages` → `New` → `Java Package`:

Package
`com.marketplace.socket`
`com.marketplace.socket.handlers`
`com.marketplace.socket.services`
`com.marketplace.socket.db`

4.4 Create Main Class

Right-click `com.marketplace.socket` → `New` → `Java Class` → Name: `Main`

```
package com.marketplace.socket;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Main {
    private static final int PORT = getPort();

    private static int getPort() {
        String portEnv = System.getenv("SOCKET_SERVER_PORT");
        if (portEnv != null && !portEnv.isEmpty()) {
            return Integer.parseInt(portEnv);
        }
        return 9090; // default
    }

    public static void main(String[] args) {
        System.out.println("Marketplace Socket Server Starting...");
        System.out.println("Port: " + PORT);

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Socket Server listening on port " + PORT);

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("New client connected: " +
clientSocket.getInetAddress());

                // TODO: Create a new thread for each client
                // new Thread(new ClientHandler(clientSocket)).start();
            }
        } catch (IOException e) {
            System.err.println("Server error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

4.5 Create Database Connection Class (Placeholder)

`com.marketplace.socket.db.DatabaseConnection.java`

```java
package com.marketplace.socket.db;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    private static final String DB_HOST = getEnv("DB_HOST", "localhost");
    private static final String DB_PORT = getEnv("DB_PORT", "26257");
    private static final String DB_NAME = getEnv("DB_NAME",
"marketplace_db");
    private static final String DB_USER = getEnv("DB_USER", "root");
    private static final String DB_PASSWORD = getEnv("DB_PASSWORD", "");

    private static String getEnv(String key, String defaultValue) {
        String value = System.getenv(key);
        return (value != null && !value.isEmpty()) ? value : defaultValue;
    }

    private static String getJdbcUrl() {
        return String.format("jdbc:postgresql://%s:%s/%s?sslmode=disable",
                DB_HOST, DB_PORT, DB_NAME);
    }

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName("org.postgresql.Driver");
            return DriverManager.getConnection(getJdbcUrl(), DB_USER,
DB_PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("PostgreSQL JDBC Driver not found", e);
        }
    }

    // Test connection (remove later)
    public static void main(String[] args) {
        try (Connection conn = getConnection()) {
            System.out.println("Database connection successful!");
        } catch (SQLException e) {
            System.err.println("Database connection failed: " +
e.getMessage());
        }
    }
}
```

4.6 Create Skeleton Handler

`com.marketplace.socket.handlers.AuthHandler.java`

```java
package com.marketplace.socket.handlers;

public class AuthHandler {
```

```
    public String handleRegister(String jsonPayload) {
        // TODO: Parse JSON, create user, return OK|{...} or ERROR|{...}
        return "ERROR|{\"code\":\"501\",\"message\":\"Not implemented
yet\"}";
    }

    public String handleLogin(String jsonPayload) {
        return "ERROR|{\"code\":\"501\",\"message\":\"Not implemented
yet\"}";
    }
}
```

4.7 Build Socket Server

Right-click `MarketplaceSocketServer` → **Clean and Build**

---

5. Spring Boot Web Services Project

5.1 Create Project using Spring Initializr

Option A: Download from start.spring.io (Recommended)

1. Open https://start.spring.io
2. Configure:

Field	Value
Project	Maven
Language	Java
Spring Boot	3.2.5 or 3.3.x
Group	`com.marketplace`
Artifact	`MarketplaceWebServices`
Name	`MarketplaceWebServices`
Package name	`com.marketplace.webservices`
Packaging	Jar
Java	21

3. Dependencies to add:
   - **Spring Web**
   - **Spring Boot DevTools**
4. Click **Generate** → Download ZIP
5. Extract ZIP to:
   `C:\Users\YourName\Documents\OnlineMarketplace\MarketplaceWebServices`

6. In NetBeans: `File` → `Open Project` → Select the extracted folder

Option B: Create in NetBeans (if Spring Boot plugin is installed)

`File` → `New Project` → `Maven` → `Spring Boot Basic` → Follow wizard

5.2 Configure `pom.xml`

Add the shared models dependency:

```
<dependency>
    <groupId>com.marketplace</groupId>
    <artifactId>MarketplaceSharedModels</artifactId>
    <version>1.0-SNAPSHOT</version>
</dependency>
```

Add Gson for JSON (if not already present):

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
</dependency>
```

Add dotenv for environment variables:

```
<dependency>
    <groupId>io.github.cdimascio</groupId>
    <artifactId>dotenv-java</artifactId>
    <version>3.0.0</version>
</dependency>
```

Full `pom.xml` (relevant sections):

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>

    <groupId>com.marketplace</groupId>
    <artifactId>MarketplaceWebServices</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>MarketplaceWebServices</name>
```

```
    <properties>
        <java.version>21</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!-- Shared Models -->
        <dependency>
            <groupId>com.marketplace</groupId>
            <artifactId>MarketplaceSharedModels</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>

        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.10.1</version>
        </dependency>

        <dependency>
            <groupId>io.github.cdimascio</groupId>
            <artifactId>dotenv-java</artifactId>
            <version>3.0.0</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

5.3 Configure `application.properties`

`src/main/resources/application.properties`:

```
# Server Configuration
server.port=${WEB_SERVICES_PORT:8080}

# Socket Server Connection
```

```
socket.server.host=${SOCKET_SERVER_HOST:localhost}
socket.server.port=${SOCKET_SERVER_PORT:9090}

# Logging
logging.level.com.marketplace=DEBUG
```

5.4 Create Package Structure

Inside `src/main/java/com/marketplace/webservices/`:

Right-click → New → Java Package:

Package
`com.marketplace.webservices.controllers`
`com.marketplace.webservices.services`
`com.marketplace.webservices.dto`

5.5 Create Socket Forwarder Service

`com.marketplace.webservices.services.SocketForwarder.java`

```
package com.marketplace.webservices.services;

import com.google.gson.Gson;
import com.google.gson.JsonObject;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;

@Service
public class SocketForwarder {

    @Value("${socket.server.host:localhost}")
    private String socketHost;

    @Value("${socket.server.port:9090}")
    private int socketPort;

    private final Gson gson = new Gson();

    public String sendCommand(String command, JsonObject payload) {
        String message = command + "|" + gson.toJson(payload);

        try (Socket socket = new Socket(socketHost, socketPort);
             PrintWriter out = new PrintWriter(socket.getOutputStream(),
true);
             BufferedReader in = new BufferedReader(new
```

```
InputStreamReader(socket.getInputStream()))) {

            out.println(message);
            return in.readLine();

        } catch (Exception e) {
            return "ERROR|{\"code\":\"500\",\"message\":\"Socket connection
failed: " + e.getMessage() + "\"}";
        }
    }
}
```

5.6 Create REST Controllers

`com.marketplace.webservices.controllers.AuthController.java`

```
package com.marketplace.webservices.controllers;

import com.google.gson.JsonObject;
import com.marketplace.webservices.services.SocketForwarder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class AuthController {

    @Autowired
    private SocketForwarder socketForwarder;

    @PostMapping("/register")
    public ResponseEntity<String> register(@RequestBody String requestBody)
{
        JsonObject payload = new JsonObject();
        // Parse requestBody and build payload
        // Forward to socket server
        String response = socketForwarder.sendCommand("REGISTER", payload);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody String requestBody) {
        JsonObject payload = new JsonObject();
        String response = socketForwarder.sendCommand("LOGIN", payload);
        return ResponseEntity.ok(response);
    }
}
```

`com.marketplace.webservices.controllers.DepositController.java`

```
package com.marketplace.webservices.controllers;

import com.google.gson.JsonObject;
```

```java
import com.marketplace.webservices.services.SocketForwarder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class DepositController {

    @Autowired
    private SocketForwarder socketForwarder;

    @PostMapping("/deposit")
    public ResponseEntity<String> deposit(@RequestHeader("Authorization")
String token,
                                          @RequestBody String requestBody) {
        JsonObject payload = new JsonObject();
        payload.addProperty("token", token.replace("Bearer ", ""));
        // Parse requestBody for amount
        String response = socketForwarder.sendCommand("DEPOSIT", payload);
        return ResponseEntity.ok(response);
    }
}
```

com.marketplace.webservices.controllers.PartnerStoreController.java

```java
package com.marketplace.webservices.controllers;

import com.google.gson.JsonObject;
import com.marketplace.webservices.services.SocketForwarder;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/store")
public class PartnerStoreController {

    @Autowired
    private SocketForwarder socketForwarder;

    @GetMapping("/products")
    public ResponseEntity<String> getProducts(@RequestHeader("X-API-KEY")
String apiKey) {
        JsonObject payload = new JsonObject();
        payload.addProperty("apiKey", apiKey);
        String response = socketForwarder.sendCommand("GET_PRODUCTS",
payload);
        return ResponseEntity.ok(response);
    }

    @PostMapping("/order")
    public ResponseEntity<String> placeOrder(@RequestHeader("X-API-KEY")
String apiKey,
                                             @RequestBody String
requestBody) {
```

```
        JsonObject payload = new JsonObject();
        payload.addProperty("apiKey", apiKey);
        // Parse requestBody for item_id, customer_reference, customer_email
        String response = socketForwarder.sendCommand("PLACE_ORDER",
payload);
        return ResponseEntity.ok(response);
    }
}
```

5.7 Update Main Application Class

`MarketplaceWebServicesApplication.java` (auto-generated)

```
package com.marketplace.webservices;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MarketplaceWebServicesApplication {

    public static void main(String[] args) {
        SpringApplication.run(MarketplaceWebServicesApplication.class,
args);
        System.out.println("Marketplace Web Services Started on port: " +
            System.getProperty("server.port", "8080"));
    }
}
```

5.8 Build Web Services

Right-click `MarketplaceWebServices` → **Clean and Build**

---

6. Building All Projects

Correct order (due to dependencies):

```
# 1. Build Shared Models first
cd C:\Users\YourName\Documents\OnlineMarketplace\MarketplaceSharedModels
mvn clean install

# 2. Build Socket Server
cd ..\MarketplaceSocketServer
mvn clean compile

# 3. Build Web Services
cd ..\MarketplaceWebServices
mvn clean compile
```

Or in NetBeans:

1. Right-click `MarketplaceSharedModels` → `Clean and Build` → `Run Maven` → `install`
2. Right-click `MarketplaceSocketServer` → `Clean and Build`
3. Right-click `MarketplaceWebServices` → `Clean and Build`

---

7. Running the Servers

Option A: From NetBeans

Server	Action
Socket Server	Right-click `MarketplaceSocketServer` → `Run`
Web Services	Right-click `MarketplaceWebServices` → `Run`

Option B: From Command Line

```
# Terminal 1 - Socket Server
cd C:\Users\YourName\Documents\OnlineMarketplace\MarketplaceSocketServer
mvn exec:java -Dexec.mainClass="com.marketplace.socket.Main"

# Terminal 2 - Web Services
cd C:\Users\YourName\Documents\OnlineMarketplace\MarketplaceWebServices
mvn spring-boot:run
```

Option C: Run JAR files

```
# Build JARs first
cd MarketplaceSocketServer
mvn package
java -jar target/MarketplaceSocketServer-1.0-SNAPSHOT.jar

cd ..\MarketplaceWebServices
mvn package
java -jar target/MarketplaceWebServices-1.0-SNAPSHOT.jar
```

---

8. Ports Reference

Component	Port	Configurable Via
Socket Server	`9090`	`SOCKET_SERVER_PORT` env var
Spring Boot Web Services	`8080`	`WEB_SERVICES_PORT` env var
Bridge Server (existing)	`8081` *(assumed)*	N/A

Component	Port	Configurable Via
CockroachDB (future)	`26257`	`DB_PORT` env var

Check if ports are in use (Windows):

```
netstat -ano | findstr :9090
netstat -ano | findstr :8080
netstat -ano | findstr :8081
```

Kill process using a port:

```
taskkill /PID <PID> /F
```

---

9. Configuration for CI/CD + Live Deployment

Recommended Approach: Environment Variables + Fallback

This allows:

- **Local development:** Uses defaults or `.env` file
- **CI/CD pipelines:** Set env vars in GitHub Actions
- **Production deployment:** Set env vars on server

9.1 Environment Variables to Set

Variable	Purpose	Default
`SOCKET_SERVER_PORT`	Socket server listen port	`9090`
`WEB_SERVICES_PORT`	Spring Boot listen port	`8080`
`SOCKET_SERVER_HOST`	Host for socket client (Web Services)	`localhost`
`DB_HOST`	Database host	`localhost`
`DB_PORT`	Database port	`26257`
`DB_NAME`	Database name	`marketplace_db`
`DB_USER`	Database user	`root`
`DB_PASSWORD`	Database password	(empty)
`JWT_SECRET`	Secret key for JWT	(must set in prod)

9.2 Local Development with `.env` (Optional)

Create `.env` file in project root (DO NOT commit to git):

```
# Socket Server
SOCKET_SERVER_PORT=9090

# Web Services
WEB_SERVICES_PORT=8080
SOCKET_SERVER_HOST=localhost

# Database (future)
DB_HOST=localhost
DB_PORT=26257
DB_NAME=marketplace_db
DB_USER=root
```

Load in code using `dotenv-java` (already added to `pom.xml`):

```java
import io.github.cdimascio.dotenv.Dotenv;

Dotenv dotenv = Dotenv.load();
String port = dotenv.get("SOCKET_SERVER_PORT", "9090");
```

9.3 GitHub Actions CI/CD Example

`.github/workflows/build.yml`:

```yaml
name: Build and Test

on:
  push:
    branches: [ dev, main ]
  pull_request:
    branches: [ dev ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v4

    - name: Set up JDK 21
      uses: actions/setup-java@v4
      with:
        java-version: '21'
        distribution: 'temurin'

    - name: Build Shared Models
      run: |
        cd MarketplaceSharedModels
        mvn clean install

    - name: Build Socket Server
      run: |
```

```
        cd MarketplaceSocketServer
        mvn clean compile

    - name: Build Web Services
      run: |
        cd MarketplaceWebServices
        mvn clean compile

    - name: Run Tests (future)
      run: |
        cd MarketplaceSocketServer
        mvn test
```

9.4 Production Deployment (Future)

Environment variables to set on server:

```
export SOCKET_SERVER_PORT=9090
export WEB_SERVICES_PORT=8080
export DB_HOST=your-db-host
export DB_PORT=26257
export DB_NAME=marketplace_prod
export DB_USER=prod_user
export DB_PASSWORD=secure_password
export JWT_SECRET=your-strong-secret-key
```

---

10. Troubleshooting

10.1 NetBeans Issues

Problem	Solution
Cannot find JDK	`Tools` → `Java Platforms` → Add JDK 21
Maven dependencies not downloading	Right-click project → `Reload Project`
Shared models not found	Run `mvn install` on SharedModels first
Spring Boot project won't open	Use `File` → `Open Project` (not Import)

10.2 Compilation Errors

Error	Solution
`package com.marketplace.models does not exist`	Build SharedModels first with `mvn install`
`cannot find symbol`	Check import statements match package names

Error	Solution
`Spring Boot version not found`	Update Spring Boot version in `pom.xml`

10.3 Runtime Errors

Error	Solution
`Address already in use`	Change port via env var or kill existing process
`Connection refused`	Start Socket Server before Web Services
`ClassNotFoundException: org.postgresql.Driver`	Add PostgreSQL dependency to `pom.xml`

10.4 Port Conflicts (Windows)

```
# Find what's using port 9090
netstat -ano | findstr :9090

# Kill the process (replace 1234 with actual PID)
taskkill /PID 1234 /F

# Or change port
set SOCKET_SERVER_PORT=9091
```

10.5 Clean Everything and Rebuild

```
# Clean all projects
cd MarketplaceSharedModels
mvn clean

cd ../MarketplaceSocketServer
mvn clean

cd ../MarketplaceWebServices
mvn clean

# Rebuild in order
cd ../MarketplaceSharedModels
mvn install

cd ../MarketplaceSocketServer
mvn compile

cd ../MarketplaceWebServices
mvn compile
```

Next Steps After Initialization

1. **Database:** Decide cloud vs local, then implement `DatabaseConnection.java`
2. **Socket Server:** Implement all handlers (Auth, Items, Purchase, etc.) with business logic
3. **Web Services:** Complete REST endpoint implementations
4. **Testing:** Test each feature using Postman and telnet
5. **CI/CD:** Set up GitHub Actions workflow
6. **Deployment:** Deploy to cloud VM or container (Docker)